

Face Hugger - description technique des fonctions cryptographiques

Projet de pièce jointe technique - à relire et valider par le responsable du produit avant tout dépôt.

Date de rédaction : 30 septembre 2026. Périmètre : édition macOS destinée au Mac App Store, version **1.0**, numéro de compilation **3**. Ce document décrit des éléments techniques constatés ; il ne propose aucune qualification juridique, exemption ou conclusion réglementaire. Aucune transmission administrative n'a été effectuée dans le cadre de sa rédaction.

1. Identification et usage du produit

Élément	Description
Nom	Face Hugger
Identifiant de l'application	dev.zakkeown.FaceHugger
Version examinée	1.0, compilation 3, configuration STORE_BUILD
Plateforme	macOS, architectures Apple Silicon / arm64 et Intel / x86_64
Code source public	< https://github.com/zak-keown/face-hugger >
Fonction principale	Interface graphique native permettant de préparer, envoyer et reprendre des téléversements volumineux vers Hugging Face Hub
Fonctions complémentaires	Consultation des dépôts et fichiers distants, création de dépôts, suppression explicite de fichiers, filtrage des fichiers locaux et gestion d'une file de téléversements

La cryptographie intervient principalement dans la protection des communications réseau et dans la protection du jeton d'authentification au moyen des services du système d'exploitation. Face Hugger n'est pas présenté comme un outil général de chiffrement de fichiers, de messagerie chiffrée, de VPN ou de gestion de clés cryptographiques.

Un dépôt « privé » correspond à un contrôle d'accès exercé par le service Hugging Face ; cette option n'est pas décrite ici comme un chiffrement de bout en bout. Le service distant reçoit les contenus destinés au téléversement. Le comportement de stockage et de chiffrement des infrastructures tierces ne fait pas partie des constatations de ce document.

2. Architecture logicielle

L'interface est écrite en Swift / SwiftUI. Elle exécute un programme intermédiaire Python embarqué, Resources/bridge.py, pour les opérations Hugging Face. Celui-ci utilise le SDK officiel et, pour les téléversements, lance la commande officielle dans un second processus Python : python -m huggingface_hub.cli hf upload

L'édition Store contient les interpréteurs et dépendances pour les deux architectures dans UploadRuntime.bundle. Elle n'installe pas ce runtime à partir d'Internet lors de son utilisation. La configuration de cette édition est distincte de celle de la version de développement à runtime téléchargé ; les constatations ci-dessous portent sur l'édition Store.

Couche	Versions / composants examinés
Interpréteur	CPython 3.12.14 ; distribution python-build-standalone du 20260901
Client Hugging Face	huggingface_hub 2.0.0
HTTP Python	httpx2 2.13.1, httpcore2 2.13.1
TLS Python	Module ssl de CPython et OpenSSL 3.5.8, daté du 25 août 2026
Confiance des certificats Python	truststore 0.10.4, contexte client utilisant par défaut le magasin de confiance du système
Transfert Xet	hf-xet / hf_xet 1.6.0, extension native Rust embarquée
Services macOS	Security.framework pour le Trousseau ; sandbox et accès aux dossiers sélectionnés par l'utilisateur

Les communications entre les processus locaux utilisent des arguments de commande, des variables d'environnement et des sorties JSON par lignes. Ce canal local n'est pas présenté comme un protocole réseau chiffré. Les autorisations de dossier sont acquises par sélection de l'utilisateur et conservées au moyen de signets d'accès sécurisé macOS.

3. Circulation des données

1. L'utilisateur sélectionne un dossier local et un dépôt Hugging Face. L'application examine les chemins, tailles et règles de sélection ; le transfert ultérieur lit les fichiers sélectionnés.
2. L'application lit le jeton d'accès Hugging Face dans le Trousseau et le transmet au processus de travail via la variable HF_TOKEN, sans le placer dans la ligne de commande.
3. Le SDK effectue les appels de service nécessaires : identité, métadonnées de dépôt, arbre de fichiers, création ou suppression demandée et préparation du transfert.
4. Les contenus sélectionnés sont transmis aux services Hugging Face / Xet et, selon les réponses du service, aux destinations de stockage associées. Le point d'entrée standard du SDK est <https://huggingface.co>. Les destinations de transfert peuvent être fournies dynamiquement ; une liste exhaustive et immuable de noms d'hôtes n'a pas été établie.
5. Les processus renvoient à l'interface leur état et leurs journaux. Le code applique une suppression du jeton connu dans les sorties. Des informations de file d'attente, signets de dossier et caches de reprise subsistent localement ; le document ne leur attribue pas un chiffrement applicatif distinct.

Le trafic réseau sortant est autorisé par la sandbox. L'application n'a pas pour fonction d'exposer un serveur TLS. Le code de lancement Store fixe notamment HF_HOME dans l'espace de données de l'application et désactive la télémétrie ainsi que la recherche de mises à jour du client Hugging Face. Ces réglages ne constituent pas une affirmation que toutes les infrastructures tierces s'abstiennent de journaliser les demandes.

4. Gestion des identifiants et des secrets

Le jeton Hugging Face est conservé comme mot de passe générique dans le Trousseau macOS, avec le service dev.zakkeown.FaceHugger, le compte logique huggingface et l'attribut kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly lors de sa création. L'utilisateur peut remplacer ou effacer ce jeton dans l'application.

Le jeton existe en mémoire et dans l'environnement du processus de travail pendant l'opération. Il ne s'agit pas d'une clé de chiffrement de fichier ni d'une clé privée de certificat client gérée par Face Hugger. L'application ne détermine pas elle-même les algorithmes ou longueurs de clés internes du Trousseau ; ces propriétés dépendent de macOS et du matériel et ne sont pas certifiées par le présent document.

Le code de l'application ne fournit pas de réglage permettant à l'utilisateur de choisir une suite cryptographique TLS, de définir une clé de session ou de désactiver la vérification des certificats. L'absence d'une telle interface n'implique pas l'absence de toutes les options correspondantes dans les bibliothèques tierces.

5. TLS de la couche Python : observations et limites

Le code installé de `httpx2/_config.py` utilise par défaut `truststore.SSLContext(ssl.PROTOCOL_TLS_CLIENT)` lorsque la vérification est activée. Le contexte par défaut a été instancié localement avec le runtime arm64 embarqué, sans connexion réseau, avec l'option Python `-B` afin de ne pas modifier le runtime signé.

Résultats de cette observation :

Propriété	Valeur observée
Bibliothèque	OpenSSL 3.5.8 25 Aug 2026
Version minimale du contexte	TLS 1.2
Version maximale du contexte	MAXIMUM_SUPPORTED ; ne constitue pas une version négociée
Vérification du certificat	CERT_REQUIRED
Vérification du nom du serveur	Activée (<code>check_hostname = True</code>)

Les suites suivantes sont **annoncées par ce contexte local**, pas constatées sur une connexion avec un serveur :

Famille	Suites retournées par <code>get_ciphers()</code>
TLS 1.3	TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_GCM_SHA256
TLS 1.2, ECDHE + GCM	ECDHE-ECDSA-AES256-GCM-SHA384, ECDHE-RSA-AES256-GCM-SHA384, ECDHE-ECDSA-AES128-GCM-SHA256, ECDHE-RSA-AES128-GCM-SHA256
TLS 1.2, ECDHE + ChaCha20	ECDHE-ECDSA-CHACHA20-POLY1305, ECDHE-RSA-CHACHA20-POLY1305
TLS 1.2, ECDHE + AES/SHA	ECDHE-ECDSA-AES256-SHA384, ECDHE-RSA-AES256-SHA384, ECDHE-ECDSA-AES128-SHA256, ECDHE-RSA-AES128-SHA256
TLS 1.2, DHE	DHE-RSA-AES256-GCM-SHA384, DHE-RSA-AES128-GCM-SHA256, DHE-RSA-AES256-SHA256, DHE-RSA-AES128-SHA256

Pour ces suites, le contexte indique une force symétrique de 128 bits pour AES-128 et de 256 bits pour AES-256 et ChaCha20. Ces valeurs ne décrivent ni la taille d'une clé RSA de certificat, ni celle d'un groupe d'échange de clés, ni une session effectivement négociée. Aucune mesure de négociation TLS distante ou

de certificat serveur n'est jointe ici. La liste ne représente pas non plus tous les algorithmes que la bibliothèque OpenSSL pourrait prendre en charge dans d'autres configurations.

Tableau de travail pour la rubrique B3.4

Ce tableau décrit les capacités du **contexte Python par défaut observé**, et non une session distante. Les champs symmetric, digest, kea, auth et strength_bits de get_ciphers() ont été examinés. Il ne doit pas être recopié comme un inventaire exhaustif des algorithmes actifs de la couche Xet.

Algorithme / mécanisme	Mode / association constatée	Longueur vérifiée ou limite	Fonction
AES	GCM, AEAD	128 ou 256 bits de force symétrique rapportée	Confidentialité et intégrité des enregistrements TLS
ChaCha20	Poly1305, AEAD	256 bits de force symétrique rapportée	Confidentialité et intégrité des enregistrements TLS
AES	CBC, suites TLS 1.2 associées à SHA-256 ou SHA-384	128 ou 256 bits de force symétrique rapportée	Confidentialité ; intégrité assurée dans la suite TLS associée
SHA-256 / SHA-384	Fonctions de hachage indiquées dans les suites ; champ digest explicite pour les suites CBC	256 / 384 bits désignent la sortie du hachage, pas une longueur de clé de chiffrement	Opérations d'intégrité et dérivation prévues par la suite TLS ; ne pas confondre suffixe de suite et primitive AEAD
ECDHE / DHE	Échange éphémère, indiqué pour les suites TLS 1.2 énumérées	Groupes et tailles effectivement négociés non mesurés	Établissement du secret partagé
RSA / ECDSA	Authentification indiquée pour les suites TLS 1.2 énumérées	Taille RSA et courbe ECDSA non mesurées	Authentification du serveur dans le protocole TLS

Pour TLS 1.3, l'énumération des suites n'identifie pas à elle seule le groupe d'échange ni l'algorithme de signature du certificat. Aucune valeur supplémentaire n'est supposée pour ces paramètres. Le Trousseau macOS reste un service de stockage de secret du système ; aucun algorithme ni dimension de clé interne n'est attribué au produit sans justificatif distinct.

6. TLS de la couche Xet / Rust

Les SBOM CycloneDX fournis dans les deux wheels hf_xet ont été examinés et rapprochés des archives sources exactes. Les composants suivants y figurent :

Composant	Version
reqwest	0.13.2
hyper-rustls	0.27.7
tokio-rustls	0.26.4
rustls	0.23.37
rustls-pki-types	1.14.0
rustls-webpki	0.103.13
rustls-platform-verifier	0.6.2

Composant	Version
aws-lc-rs / aws-lc-sys	1.16.2 / 0.39.0
ring	0.17.14
security-framework / security-framework-sys	3.7.0 / 2.17.0

Dans la source Xet correspondant à la version 1.6.0, les modules `xet_pkg`, `xet_data` et `xet_client` activent `rustls-tls` par défaut. Cette option active `reqwest/rustls`. Dans la source exacte de `reqwest 0.13.2`, l'option `rustls` sélectionne notamment le fournisseur AWS-LC et `rustls-platform-verifier`. Il s'agit d'éléments vérifiables de configuration source ; l'inventaire SBOM seul ne prouve pas que chaque composant est utilisé pour chaque connexion.

La source du fournisseur AWS-LC dans `rustls 0.23.37` contient notamment les suites TLS 1.3 `TLS13_AES_256_GCM_SHA384`, `TLS13_AES_128_GCM_SHA256` et `TLS13_CHACHA20_POLY1305_SHA256`. Leur présence constitue une capacité de bibliothèque, pas une trace d'exécution de Face Hugger. Les fonctions de compilation, le fournisseur effectivement actif dans le binaire livré et les suites négociées en production ne sont pas attestés par une capture TLS dans ce document. La présence de `ring` dans le SBOM n'est pas utilisée pour affirmer que ce composant est le fournisseur TLS actif.

Le SBOM contient également, entre autres, `blake3 1.8.3`, `sha2 0.11.0` et `chacha20 0.10.0`. Leur inventaire est conservé pour ne pas limiter abusivement la description aux seules bibliothèques nommées « TLS ». Aucun usage spécifique de chaque primitive, ni aucune fonction de chiffrement de fichiers, n'est déduit de son seul nom dans le SBOM. Aucune certification FIPS n'est revendiquée.

7. Traçabilité des composants examinés

Les archives de crates ont été vérifiées par SHA-256 à la fois contre les valeurs du SBOM et contre les enregistrements de version du registre `crates.io`. Les archives GitHub sont identifiées par commit exact et par empreinte de leurs octets conservée lors de l'examen ; cela ne constitue pas une reproduction du binaire depuis les sources.

Élément	SHA-256
Verrou de dépendances <code>Resources/runtime-requirements.txt</code>	38b0f4fc6c2ac88f13a86e47acd810a71e6a071bd0b7532ed36172b82df728e5
Archive source Xet, commit <code>de71453d952bd8b806edaa997c72313051a49050</code>	97109eb3c5ea9b29685ef3543248f23eee204e912d716e442018c79fcbf666cc
Archive crate <code>reqwest 0.13.2</code>	ab3f43e3283ab1488b624b44b0e988d0acea0b3214e694730a055cb6b2efa801
Archive crate <code>rustls 0.23.37</code>	758025cb5fccfd3bc2fd74708fd4682be41d99e5dff73c377c0646c6012c73a4
Archive crate <code>aws-lc-sys 0.39.0</code>	1fa7e52a4c5c547c741610a2c6f123f3881e409b714cd27e6798ef020c514f0a
Archive crate <code>ring 0.17.14</code>	a4689e6c2294d81e88dc6261c768b63bc4fcd852be6d1352498b114f61383b7

Pièces et chemins du dépôt permettant la vérification :

- Resources/ThirdParty/python-packages/hf-xet-1.6.0/arm64-hf_xet.cyclonedx.json et x86_64-hf_xet.cyclonedx.json : SBOM fournis dans les wheels.
- Resources/ThirdParty/hf-xet-dependencies/provenance.json : versions, sources, checksums et notices du périmètre Xet.
- Resources/ThirdParty/python-build-standalone-20260901/provenance.json, arm64-PYTHON.json et x86_64-PYTHON.json : distribution Python et dépendances natives, dont OpenSSL.
- Resources/ThirdParty/python-packages/inventory.json et Resources/runtime-requirements.txt : inventaire Python et verrou des dépendances.
- Sources/FaceHugger/Services.swift, Resources/bridge.py, Resources/Store.entitlements et project-store.yml : lancement, identifiants, autorisations et identification de la configuration Store.
- docs/third-party-notices.md : méthode et limites de l'examen des composants tiers.

Source primaire Xet :

<<https://github.com/huggingface/xet-core/tree/de71453d952bd8b806edaa997c72313051a49050>>. Les URL exactes des archives crates.io et leurs empreintes figurent dans le manifeste de provenance ; elles ne doivent pas être remplacées par des versions plus récentes lors de la vérification de cette version du produit.

8. Points à compléter avant validation du dossier

- Paquet signé examiné : FaceHugger-1.0-3.pkg, SHA-256 5f1e5dd94055b7246c31128a677fdb6bfdea5297788c260cdadb6f8da60dffa6. Le code et les ressources ont été conservés dans le commit 7893cb6d487017ceae1fbfc7c28dcce3e4c00478; les corrections CI ultérieures ne modifient pas ce paquet. Vérifier cette empreinte avant de joindre le binaire.
- Faire valider par le responsable du produit les descriptions fonctionnelles et le périmètre territorial / commercial dans le formulaire administratif séparé.
- Si une liste exhaustive d'algorithmes effectivement activés dans Xet est requise, compléter l'examen par les paramètres de compilation du wheel concerné et une vérification ciblée du fournisseur cryptographique ; ne pas assimiler l'ensemble du SBOM à un chemin d'exécution prouvé.
- Si des caractéristiques de session sont demandées, produire des mesures contrôlées de négociation TLS en distinguant les appels Python et les transferts Xet. Les tailles des clés de certificat, groupes d'échange, durées de session et suites négociées restent non mesurées ici.
- Revoir ce document en cas de modification du runtime, des wheels, du verrou de dépendances, du paramétrage TLS ou des fonctions du produit.

Note for the product owner: this is a factual technical draft, not a legal classification or a completed filing. No credentials, private contact details, or external submission are included.